

혼동행렬과 정밀도·재현율

네 칸짜리 성적표를 읽는 법부터 무엇을 놓쳤는지 말하는 법까지

혼동행렬 confusion_matrix ravel() TP·FN·FP·TN 정밀도 재현율 precision_score recall_score
ConfusionMatrixDisplay F1

2026년 9월 14일 (월) · 39 일차

교안 28_01 혼동행렬 기반 분류 평가 (PART 02 혼동행렬 · PART 03 정밀도와 재현율)

포스코 K-뉴딜 PDM 아카데미 · 박민호

오늘 다룬 것



오늘의 숫자	값	무엇인가
평가용 설비	200 대 중 실제 고장 44 대	엔진 1,000 행 · test_size=0.2
혼동행렬 네 칸	147 · 9 · 18 · 26	TN 정상맞힘 · FP 헛걸음 · FN 놓침 · TP 고장맞힘
정확도 · 정밀도 · 재현율	0.865 · 0.743 · 0.591	같은 모델인데 세 숫자가 전부 다릅니다
더미 모델의 재현율	0.000	정확도는 0.78 인데 고장은 한 건도 못 잡았습니다

연결

다음 시간 — 28_02 고장 예측 결과 해석. 28 장의 마지막 챕터이고, 오늘 만든 세 지표를 실제 보고서 형태로 정리하는 내용입니다.

이 책은 수업에서 실제로 돌린 코드와 그 출력을 그대로 싣고, 각 숫자가 현장에서 무슨 뜻인지까지 이어 씁니다. 먼저 기호 몇 가지만 익히고 들어가겠습니다.

뱃지 · 코드블록 기호

기호	뜻
함수 메서드 속성 개념	항목 앞의 색 뱃지가 그 이름의 정체를 알려 줍니다. 점(.) 뒤에 괄호가 붙으면 메서드, 안 붙으면 속성, 앞에 점이 없으면 함수입니다.
회색 배경	파이썬 코드입니다. 수업에서 실제로 실행한 줄을 그대로 옮겼습니다.
▶ 초록 화살표	그 코드를 돌렸을 때 실제로 찍힌 출력입니다. 지어낸 값이 아닙니다.
▲ 그림 아래 회색 글	차트 캡션입니다. 무엇을 보라는 안내까지 적혀 있습니다.

오늘 처음 나오는 도구

이름	한 줄 역할	처음 나오는 장
<code>confusion_matrix()</code>	실제와 예측을 교차해 2×2 표를 만듭니다.	1 장
<code>.ravel()</code>	2×2 배열을 한 줄(네 개)로 펴 줍니다.	1 장
<code>ConfusionMatrixDisplay</code>	혼동행렬을 색칠된 표 그림으로 그려 줍니다.	1 장
<code>precision_score()</code>	정밀도 — 고장이라 경고한 것 중 진짜 고장의 비율입니다.	3 장
<code>recall_score()</code>	재현율 — 실제 고장 중 잡아낸 비율입니다.	3 장
<code>f1_score()</code>	정밀도와 재현율을 한 숫자로 묶습니다.	5 장
<code>classification_report()</code>	세 지표를 클래스별로 한 표에 찍어 줍니다.	5 장

교안에 없지만 코드에 나오는 옵션

실습 노트북과 이 책의 코드에는 교안 슬라이드에 적히지 않은 옵션이 몇 개 섞여 있습니다. 없어도 코드는 돌아가지만, 있으면 결과가 보기 좋아집니다. 무엇을 하는 옵션인지만 짧게 적어 둡니다.

옵션	하는 일
<code>cmap="Blues"</code>	히트맵의 색 계열을 파랑 계통으로 지정합니다. 숫자가 클수록 진해집니다. 빼면 기본 색(보라 계열)으로 나옵니다.
<code>values_format="d"</code>	칸 안의 숫자를 정수로 찍습니다. 빼면 147 이 1.5e+02 처럼 지수 표기로 나오는 경우가 있습니다.
<code>display_labels=[...]</code>	네 칸의 축 이름을 0·1 대신 우리말로 바꿔 답니다. 표 자체의 값은 바뀌지 않습니다.
<code>digits=3</code>	<code>classification_report</code> 의 소수점 자리수입니다. 기본은 2 자리입니다.
<code>zero_division=0</code>	분모가 0 이 될 때 경고 대신 0 을 돌려줍니다. 더미 모델처럼 1 로 예측한 건이 아예 없을 때 필요합니다.

지난 회차에서 이어지는 것

도구	오늘 하는 일	다른 회차
<code>accuracy_score(y_test, y_pred)</code>	오늘 배울 두 지표와 나란히 놓고 비교합니다.	38 일차
<code>DummyClassifier</code>	재현율로 다시 재 보면 0 이 나온다는 것을 확인합니다.	38 일차

도구	오늘 하는 일	다른 회차
<code>RandomForestClassifier</code>	오늘 채점할 모델입니다. 새로 학습시키지 않고 그대로 씁니다.	38 일차
<code>.predict_proba()</code>	경계를 옮겨 보는 절에서 다시 꺼내 씁니다.	38 일차
<code>train_test_split(..., stratify=y)</code>	평가용 200 대를 만드는 데 쓰인 그 분리입니다.	37 일차

CONTENTS

목차

오늘 배운 것의 지도

장	제목	무엇을 배우나
0 장	오늘의 지도	양성과 음성을 먼저 정합합니다. 이 한 줄을 거꾸로 잡으면 오늘 배우는 모든 숫자가 뒤집힙니다.
1 장	혼동행렬 — 네 칸짜리 성적표	<code>confusion_matrix</code> 로 표를 만들고 <code>ravel</code> 로 네 칸을 꺼냅니다. 순서가 전부입니다.
2 장	네 칸을 현장의 언어로	통계 용어를 정비팀이 읽는 문장으로 옮깁니다. 어느 칸이 가장 비싼지 따집니다.
3 장	정밀도와 재현율	개수 대신 비율로 봅니다. 두 값이 왜 한쪽을 올리면 다른 쪽이 내려가는지 확인합니다.
4 장	세 지표를 한 화면에	정확도·정밀도·재현율을 나란히 놓고 이 모델이 어느 쪽으로 치우쳤는지 한 문장으로 말합니다.
5 장	F1 과 <code>classification_report</code>	오늘 코드로는 쓰지 않았지만 슬라이드에서 지나간 두 가지를 미리 정리해 둡니다.
부록 A	치트시트	오늘 나온 도구와 공식을 한 표로 압축했습니다.
부록 B	오류·함정 사전	실습 코드에서 잡아낸 조용한 오류 하나와, 틀리기 쉬운 자리들입니다.
부록 C	실습 하이라이트 & 다음 시간 예고	오늘 손으로 짤 것 과 28_02 예고입니다.

CHAPTER 0 오늘의 지도

양성·음성을 정하는 일부터 시작합니다

38 일차에 우리는 정확도 86.5%라는 숫자 하나를 얻었습니다. 그리고 그 숫자가 **혼자서는 아무 뜻도 없다**는 것까지 확인했습니다. 무조건 정상이라고만 찍는 더미 모델도 78%가 나왔으니까요. 오늘은 그 다음 질문으로 갑니다. **그러면 무엇을 봐야 할까요.**

답은 **한 숫자를 네 칸으로 쪼개는 것**입니다. 200 대를 맞혔다/틀렸다 둘로만 나누지 않고, 맞힌 것도 두 종류로, 틀린 것도 두 종류로 나눕니다. 그러면 같은 86.5%라도 **무엇을 맞히고 무엇을 틀렸는지**가 보입니다.

오늘의 흐름



양성과 음성 — 이것부터 정합니다

혼동행렬을 배우기 전에 딱 한 가지를 먼저 못박아야 합니다. **무엇을 양성(positive)이라 부를 것인가**입니다. 그리고 답은 직관과 조금 어긋납니다.

개념

양성은 좋은 쪽이 아니라 찾아내려는 쪽입니다. 코로나 검사를 떠올리면 쉽습니다. 바이러스가 있으면 양성이라고 부르지, 건강한 쪽을 양성이라고 하지 않습니다. 설비도 똑같습니다. **우리가 찾아내려는 것은 고장 임박**이므로 `failure_soon = 1` 이 양성이고, 정상인 0 이 음성입니다.

이것을 거꾸로 잡는 것이 이 단원에서 **초보자가 가장 많이 하는 실수**입니다. 정상을 양성으로 두고 채점하면, 앞으로 배울 정밀도와 재현율이 전부 "정상을 얼마나 잘 맞혔는가"를 재게 됩니다. 정상은 원래 78%가 정상이라 아무 모델이나 잘 맞습니다. **정작 알고 싶었던 것은 하나도 못 재는 표**가 나오는 셈입니다.

네 칸의 이름과 값어치 (개념 도식)



▲ 네 칸의 이름과 값어치 — 같은 틀임이라도 왼쪽 아래(FN)와 오른쪽 위(FP)의 무게가 다릅니다 (개념 도식)

오늘 쓰는 용어

용어	기호	우리 데이터에서	한 줄 뜻
양성 (Positive)	1	failure_soon = 1	찾아내려는 쪽 — 고장이 임박한 설비
음성 (Negative)	0	failure_soon = 0	찾아낼 필요 없는 쪽 — 정상 설비
진양성	TP	26 대	고장이라 했는데 진짜 고장이었습니다.
진음성	TN	147 대	정상이라 했는데 진짜 정상이었습니다.
위양성	FP	9 대	고장이라 했는데 멀쩡했습니다. 헛걸음입니다.
위음성	FN	18 대	정상이라 했는데 고장이었습니다. 놓쳤습니다.

앞의 진(True)·위(False)는 모델이 맞혔는지를, 뒤의 양성·음성은 모델이 뭐라고 답했는지를 가리킵니다. 그래서 FN 은 "모델이 음성(정상)이라 했는데 그게 틀렸다", 즉 **고장을 정상이라고 한 경우**입니다. 이 조합 규칙만 잡으면 네 이름을 외울 필요가 없습니다.

주의

이름에서 가장 헷갈리는 지점이 여기입니다. **FN의 N은 실체가 아니라 모델의 답**입니다. 실제로는 고장인데 모델이 정상이라고 답했으니 "위(틀린) 음성"입니다. 실제 기준으로 읽으면 정반대로 이해하게 되므로, 뒤 글자는 항상 **예측**이라고 새기십시오.

CHAPTER 1 혼동행렬

네 칸짜리 성적표를 만들고 꺼내기

38 일차 마지막에 우리는 모델과 더미의 정확도 차이가 크지 않다는 것을 확인했습니다. 그런데 **왜 차이가 안 나는지**는 정확도로는 알 수 없었습니다. 이제 그 안을 열어 봅시다.

1.1 200 대를 네 칸에 나눠 담기

평가용 200 대는 각각 **실제로 고장이었는가**와 **모델이 고장이라 했는가** 두 가지 답을 가지고 있습니다. 예/아니오가 두 개니까 조합은 네 가지입니다. **모든 설비는 이 네 칸 중 정확히 한 칸에 들어가고**, 따라서 네 칸의 합은 항상 평가한 전체 대수가 됩니다.

함수 `confusion_matrix()` 실제와 예측을 교차한 2×2 표

정의 정답 배열과 예측 배열을 받아 **네 칸이 채워진 2×2 배열**을 돌려줍니다.

형태 `cm = confusion_matrix(y_test, y_pred)`

38 일차에 학습시킨 모델과 그 예측을 그대로 씁니다. 새로 학습시키지 않습니다.

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(cy_test, cy_pred)  # (정답, 예측) 순서 — accuracy_score 와 같습니다
print(cm)

▶ [[147   9]
   [ 18  26]]
```

대괄호가 두 겹이고 숫자 사이에 쉼표가 없습니다. **쉼표가 없다는 것은 파이썬 리스트가 아니라 넘파이 배열**이라는 신호입니다. 리스트였다면 `[[147, 9], [18, 26]]`처럼 쉼표가 찍힙니다.

평가용 200대의 혼동행렬

실제	정상(0) -	147 TN 정상을 정상	9 FP 헛걸음
	고장 임박(1) -	18 FN 놓침	26 TP 제대로 잡음
		정상(0)	고장 임박(1)
		모델의 예측	

▲ 평가용 200 대의 실제 혼동행렬 — 진한 칸이 개수가 많은 칸이고, 왼쪽 아래 18 이 오늘 우리가 줄여야 할 숫자입니다

왜 쓰나 정확도 하나로는 틀린 27 건이 어느 쪽 틀림인지 알 수 없습니다. 여기서는 9 건이 헛걸음, 18 건이 놓침이라는 것이 바로 보입니다. 27 건이라는 같은 숫자가 9 대 18 이냐 18 대 9 나에 따라 현장에서는 완전히 다른 이야기가 됩니다.

설비 적용

네 칸의 합(147+9+18+26)이 평가용 200 대와 맞는지 먼저 확인하는 습관을 들이십시오. 합이 안 맞으면 `y_test` 와 `y_pred` 의 길이가 어긋난 것이고, 그 상태로 계산한 지표는 전부 무의미합니다.

주의

인자 순서는 (정답, 예측)입니다. `accuracy_score` 는 두 배열이 같은지만 세기 때문에 순서를 바꿔도 값이 같았지만, `confusion_matrix` 는 순서를 바꾸면 표가 대각선 기준으로 뒤집힙니다. 9 와 18 의 자리가 바뀌어 놓침과 헛걸음이 통째로 맞바뀝니다.

1.2 표를 읽는 방향

scikit-learn 은 라벨을 오름차순으로 배치합니다. 우리 라벨은 0 과 1 이므로 행도 0·1, 열도 0·1 순서입니다. 그래서 표는 아래처럼 읽힙니다.

	예측 0 (정상)	예측 1 (고장 임박)	행 합계
실제 0 (정상)	147 TN	9 FP	156 대
실제 1 (고장 임박)	18 FN	26 TP	44 대
열 합계	165 대	35 대	200 대

행 합계가 실제로 그랬던 대수입니다. 실제 고장이 44 대, 실제 정상이 156 대입니다. 열 합계는 모델이 그렇게 답한 대수로, 모델은 35 대만 고장이라고 경고했습니다. 44 대가 실제인데 35 대만 경고했으니 애초에 산술적으로 최소 9 대는 놓칠 수밖에 없는 상황이었습니다.

개념

대각선(147 과 26)이 **맞힌 칸**이고, 반대 대각선(9 와 18)이 **틀린 칸**입니다. 정확도는 결국 대각선의 합을 전체로 나눈 값입니다 — $(147+26) \div 200 = 0.865$. 38 일차에 손으로 센 그 값이 여기서 다시 나옵니다.

1.3 네 칸을 변수로 꺼내기

메서드 `cm.ravel()` 2×2 배열을 한 줄로 펴기

정의 2 차원 배열을 왼쪽 위부터 오른쪽으로 읽어 1 차원으로 펴 줍니다. 2×2 면 값 네 개가 나옵니다.

형태 `tn, fp, fn, tp = cm.ravel()`

정밀도와 재현율은 네 칸을 더하고 나눠서 만드는 값입니다. 그래서 칸을 각각 변수에 담아 두면 공식을 그대로 옮겨 쓸 수 있습니다.

```
tn, fp, fn, tp = cm.ravel()

print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")
print("네 칸의 합:", tn + fp + fn + tp)
```

- ▶ `TN=147, FP=9, FN=18, TP=26`
- ▶ 네 칸의 합: 200

합이 평가용 200 대와 정확히 맞습니다. **이 확인 한 줄을 습관으로 두십시오.** 뒤에 나올 모든 지표가 이 네 값 위에 얹히기 때문에, 여기가 틀리면 아래가 전부 틀립니다.

왜 쓰나 `ravel` 은 "풀어 헤치다"라는 뜻입니다. 표 모양을 유지한 채 `cm[1][0]` 처럼 인덱스로 꺼내도 되지만, **한 줄로 네 이름을 동시에 불러올 수 있어** 이후 공식이 훨씬 읽기 쉬워집니다.

자주 하는 실수

순서는 무조건 `tn, fp, fn, tp` 입니다. 표를 왼쪽 위 → 오른쪽 위 → 왼쪽 아래 → 오른쪽 아래로 읽은 순서 그대로입니다. 외우기 어려우면 외우지 말고, 위 1.2 의 표를 옆에 띄워 두고 그때그때 확인하십시오. 이 순서를 한 글자만 바꿔 써도 오류는 안 나고 값만 조용히 뒤바뀝니다 — 실제로 일어나는 사고이고, 부록 B 에 정리해 두었습니다.

1.4 여섯 대로 손으로 채워 보기

200 대짜리 표는 눈으로 검산할 수 없습니다. 그래서 **여섯 대짜리 장난감 데이터**로 먼저 맞춰 봅니다. 정답과 예측을 직접 적어 놓고 네 칸이 제대로 나오는지 확인하는 것입니다.

```

ex_test = [1, 1, 1, 0, 0, 0] # 실제 — 앞 세 대가 고장
ex_pred = [1, 1, 0, 1, 0, 0] # 예측 — 세 번째를 놓치고 네 번째를 헛경고

ex_cm = confusion_matrix(ex_test, ex_pred)
print(ex_cm)

ex_tn, ex_fp, ex_fn, ex_tp = ex_cm.ravel()
print(f"ex_TN={ex_tn}, ex_FP={ex_fp}, ex_FN={ex_fn}, ex_TP={ex_tp}")

▶ [[2 1]
   [1 2]]
▶ ex_TN=2, ex_FP=1, ex_FN=1, ex_TP=2

```

여섯 대를 한 대씩 따라가 보면 왜 이 값이 나오는지 손으로 확인됩니다.

설비	실제	예측	어느 칸	읽는 법
1 호기	1 고장	1 고장	TP	제대로 잡았습니다.
2 호기	1 고장	1 고장	TP	제대로 잡았습니다.
3 호기	1 고장	0 정상	FN	고장인데 정상이라 했습니다 — 놓침.
4 호기	0 정상	1 고장	FP	정상인데 고장이라 했습니다 — 헛걸음.
5 호기	0 정상	0 정상	TN	정상을 정상이라 했습니다.
6 호기	0 정상	0 정상	TN	정상을 정상이라 했습니다.

TP 2, FN 1, FP 1, TN 2. 함수가 돌려준 값과 **정확히 같습니다**. 여섯 줄짜리 데이터로 한 번 맞춰 보면 200 대짜리 표도 같은 규칙으로 채워졌다는 것을 믿을 수 있습니다.

응용

새 지표를 배울 때마다 이 여섯 대 데이터를 다시 꺼내 쓰십시오. 네 칸이 **2·1·1·2**로 **대칭**이라 정밀도도 재현율도 전부 0.667 이 나오고, 어느 한쪽만 틀리게 계산하면 바로 표가 납니다. **검산용 데이터를 하나 만들어 두는** 것은 실무에서도 그대로 쓰는 습관입니다.

1.5 색으로 보기 — ConfusionMatrixDisplay

개념 ConfusionMatrixDisplay 혼동행렬을 색칠된 표로

정의 혼동행렬 배열을 받아 **칸마다 색을 입힌 그림**으로 그려 줍니다. 숫자가 클수록 진해집니다.

형태 ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[...]).plot()

숫자 네 개는 눈이 금방 지칩니다. 보고 자료에는 색이 들어간 표가 훨씬 잘 읽힙니다.

```

from sklearn.metrics import ConfusionMatrixDisplay

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["정상", "고장"])
disp.plot(cmap="Blues", values_format="d")

▶ (2×2 색 표가 그려집니다 — 147 칸이 가장 진하게 나옵니다)

```

display_labels로 축 이름을 0·1 대신 우리말로 바꿔 달았습니다. **표의 값 자체는 바뀌지 않고 보이는 이름만 바뀝니다**.

왜 쓰나 진한 칸이 어디인지가 한눈에 들어옵니다. **FN 칸이 진하면 놓침이 많다는 신호**이므로, 여러 모델을 비교할 때 이 그림 녀장을 나란히 붙여 두면 설명이 필요 없어집니다.

주의

이 그림에서 가장 진한 칸은 거의 항상 TN 입니다. 정상 설비가 원래 많기 때문입니다. **색의 진하기로 성능을 판단하면 안 됩니다**
— 색은 개수를 나타낼 뿐입니다. 봐야 할 칸은 진한 칸이 아니라 왼쪽 아래 FN 입니다.

1.6 같은 표를 비율로 보기

네 칸은 개수입니다. 그런데 **행마다 나눠서 비율로** 보면 다른 것이 보입니다. `confusion_matrix` 에는 그 자리에서 비율로 바꿔 주는 옵션이 있습니다.

```
import numpy as np

행기준 = confusion_matrix(cy_test, cy_pred, normalize="true") # 실제(행)를 1로 두고 나눔
열기준 = confusion_matrix(cy_test, cy_pred, normalize="pred") # 예측(열)을 1로 두고 나눔

print("행 기준\n", np.round(행기준, 3))
print("열 기준\n", np.round(열기준, 3))
```

- ▶ 행 기준
- ▶ `[[0.942 0.058]`
- ▶ `[0.409 0.591]]`
- ▶ 열 기준
- ▶ `[[0.891 0.257]`
- ▶ `[0.109 0.743]]`

오른쪽 아래 칸을 보십시오. **행 기준에서는 0.591, 열 기준에서는 0.743** 입니다. 3장에서 배울 재현율과 정밀도의 값이 이미 여기 들어 있습니다. 우연이 아니라 **정의가 그렇습니다** — 재현율은 실제 고장 44 대를 1로 두고 나눈 값이고, 정밀도는 경고한 35 대를 1로 두고 나눈 값입니다.

보는 방향	오른쪽 아래 칸	왼쪽 아래 칸	읽는 법
행 기준 <code>normalize="true"</code>	0.591 = 재현율	0.409 = 놓친 비율	실제 고장 44 대를 100%로 놓고 나눕니다.
열 기준 <code>normalize="pred"</code>	0.743 = 정밀도	0.257 = 헛걸음 비율	경고한 35 대를 100%로 놓고 나눕니다.

개념

그래서 **정밀도와 재현율은 새로운 개념이 아닙니다**. 같은 네 칸을 어느 방향으로 나눴느냐의 차이일 뿐입니다. 3장에서 공식을 외우려 하지 말고 이 그림을 떠올리십시오 — **가로로 나누면 재현율, 세로로 나누면 정밀도**입니다.

주의

`normalize` 는 교안에 나오지 않는 옵션입니다. "`true`"는 실제(행)를 기준으로, "`pred`"는 예측(열)을 기준으로, "`all`"은 전체를 기준으로 나눠 줍니다. **비율만 바꿀 뿐 원래 개수 정보는 사라지므로**, 보고용으로는 개수 표와 비율 표를 함께 싣는 편이 안전합니다.

1장 한 장 정리

배운 것	한 줄 정의	기억할 숫자
양성 · 음성	좋은 쪽이 아니라 찾아내려는 쪽이 양성	고장 임박 = 1 = 양성
<code>confusion_matrix(y_test, y_pred)</code>	실제와 예측을 교차해 2×2 배열을 만든다	<code>[[147 9] [18 26]]</code>

배운 것	한 줄 정의	기억할 숫자
<code>cm.ravel()</code>	왼쪽 위에서 부터 네 값으로 편다	tn, fp, fn, tp 순서
네 칸의 합	평가한 전체 대수와 반드시 같다	$147+9+18+26 = 200$
<code>ConfusionMatrixDisplay</code>	네 칸을 색 표로 그린다	진한 칸은 개수일 뿐

한 장 정리

1 장의 결론은 **정확도 하나를 네 칸으로 쪼갬**다는 것입니다. 같은 86.5%인데 틀린 27 건이 헛걸음 9 건과 놓침 18 건으로 갈라졌습니다. 다음 장에서는 이 숫자들이 현장에서 각각 얼마짜리인지를 봅니다.

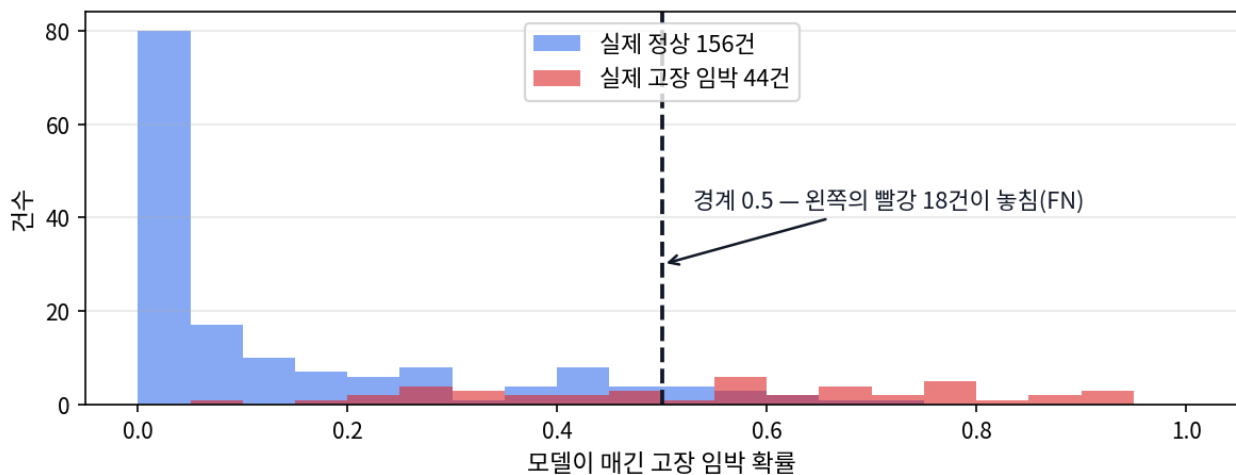
TN=147, FP=9, FN=18, TP=26을 그대로 복사해 정비팀에 메일로 보내면 어떻게 될까요. 심중팔구 되묻는 전화가 옵니다. 숫자를 현장의 문장으로 옮기는 것까지가 우리 일입니다.

2.1 FN — 놓침, 가장 비싼 칸

네 칸 중 가장 먼저 봐야 할 칸은 왼쪽 아래 FN입니다. 실제로 고장이 임박했는데 모델이 정상이라고 답한 18대입니다. 현장에서 이 18대는 이렇게 나타납니다 — 아무 경고 없이, 어느 날 갑자기 멈춥니다.

같은 고장이라도 **예측된 고장과 예측되지 않은 고장은 손실이 완전히 다릅니다**. 미리 알았다면 정비 계획에 넣고, 부품을 미리 받아 두고, 라인이 비는 시간에 교체합니다. 모르고 당하면 가동 중에 멈추고, 그때부터 부품을 찾습니다. 대형 생산 설비의 부품은 해외 발주에 리드타임이 몇 달씩 걸리는 것이 흔합니다. **그 몇 달 동안 라인이 서 있는 비용이 FN 한 건의 진짜 값입니다**.

놓친 18건 중 10건은 확률 0.3~0.49 — 경계 바로 아래에 있었습니다



▲ 실제 정상 156 건(파랑)과 실제 고장 44 건(빨강)의 확률 분포 — 경계 0.5 왼쪽에 남은 빨강 18 건이 놓침입니다

놓친 18건이 어디에 있었는지 확률로 보면 뼈아픕니다. 18건의 확률은 **0.06에서 0.49 사이**이고 중앙값이 0.32입니다. 그중 **10건은 확률 0.3 이상**, 즉 모델도 꽤 의심하고 있었는데 0.5라는 선을 못 넘어 정상으로 분류된 것들입니다. 가장 가까운 한 건은 **0.49**였습니다 — 0.01 차이로 놓쳤습니다.

```
import numpy as np

proba = c_model.predict_proba(cX_test)[: , 1]
miss = proba[(cy_pred == 0) & (cy_test.values == 1)] # 놓친 18건의 확률

print("놓친 건수:", len(miss))
print("확률 범위:", round(miss.min(), 2), "~", round(miss.max(), 2))
print("0.3 이상이었던 건:", int((miss >= 0.3).sum()), "건")
```

- ▶ 놓친 건수: 18
- ▶ 확률 범위: 0.06 ~ 0.49
- ▶ 0.3 이상이었던 건: 10 건

설비 적용

이 결과는 모델을 다시 학습시키지 않고도 놓침을 줄일 길이 있다는 뜻입니다. 판정 경계를 0.5에서 0.3으로만 내려도 이 10건이 경고 목록에 올라옵니다. 대신 헛걸음이 늘어납니다 — 그 맞바꿈을 3장에서 숫자로 봅니다.

2.2 FP — 헛걸음, 싸지만 공짜는 아닙니다

오른쪽 위 FP 9 건은 **고장이라고 경고했는데 가 보니 멀쩡했던** 설비입니다. FN 에 비하면 훨씬 가볍습니다. 아무도 다치지 않고 라인도 안 섭니다. 다만 두 가지가 발생합니다.

FP 가 만드는 것	구체적으로	쌓이면
점검 비용	사람이 가서 뜯어보는 시간과 인건비가 실제로 나갑니다.	예산 항목으로 잡힐 만큼 커집니다.
신뢰 손실	또 헛걸음이네 라는 말이 나옵니다.	진짜 경고가 왔을 때 현장이 안 움직입니다.

두 번째가 더 무섭습니다. **FP 를 0 으로 만드는 것이 목표가 아니라 적정 수준으로 관리하는 것이 목표인** 이유가 여기 있습니다. 50 대를 경고했는데 그중 12 대만 진짜였다면, 현장은 그 다음 경고부터 믿지 않습니다. 그러다 진짜가 터지면 그때는 FN 이나 마찬가지로 됩니다.

비유

응급실로 생각하면 정확합니다. 앰불런스를 불렀는데 가 보니 큰일이 아니었다면 — **출동 비용은 나갔지만 사람은 살아 있습니다.** 다행인 쪽입니다. 반대로 심각한데 부르지 않은 경우가 FN 입니다. 비용을 비교할 대상이 아닙니다.

2.3 숫자를 현장 문장으로

네 칸을 그대로 출력하지 말고, **읽는 사람이 아는 말로 한 줄씩 바꿔** 찍습니다. 수업에서 짤 출력이 이 형태였습니다.

```
tn, fp, fn, tp = cm.ravel()

print(f"제대로 잡은 고장 : {tp} 대")
print(f"놓친 고장 : {fn} 대 ← 가장 먼저 줄여야 합니다")
print(f"헛걸음(정상인데 경고): {fp} 대")
print(f"정상을 정상으로 판정 : {tn} 대")
```

- ▶ 제대로 잡은 고장 : 26 대
- ▶ 놓친 고장 : 18 대 ← 가장 먼저 줄여야 합니다
- ▶ 헛걸음(정상인데 경고): 9 대
- ▶ 정상을 정상으로 판정 : 147 대

같은 네 숫자인데 읽는 데 걸리는 시간이 다릅니다. 그리고 이렇게 써 두면 **놓친 고장 18 대**라는 말이 화면에 박히므로, 보고를 받는 사람도 자연스럽게 그 숫자부터 묻게 됩니다.

응용

보고 대상에 따라 한 번 더 번역이 필요합니다. **현장 관리자에게는 대수로, 경영진에게는 금액으로** 씁니다. "놓친 고장 18 대"가 아니라 "예방 정비로 처리할 수 있었던 계획 외 정지가 18 건 남아 있습니다"가 되고, 그다음 단계에서는 정지 1 회당 손실을 곱한 금액이 됩니다. 숫자를 읽는 사람의 언어로 옮기는 것도 이 직무의 일입니다.

2.4 어느 실수가 더 비싼가

현장에서 문제를 처리할 때 **먼저 들어온 순서대로** 처리하는 것이 가장 흔한 실수입니다. 순서가 아니라 **심각도로** 잡아야 합니다. 혼동행렬은 그 우선순위를 숫자로 만들어 줍니다.

구체적인 비용은 설비마다 다르지만, 값을 한 번 넣어 보면 감이 잡힙니다. 아래는 **계획 외 정지 1 회 2,000 만 원, 헛걸음 점검 1 회 30 만 원**이라고 가정했을 때의 계산입니다. 숫자는 가정이고, 방법이 요점입니다.

```
비용_농침 = 2000      # 만원 — 계획 외 정지 1 회 (가정)
비용_헛걸음 = 30      # 만원 — 점검 1 회 (가정)
```

```
손실 = fn * 비용_농침 + fp * 비용_헛걸음
print(f"농침 {fn}건 = {fn * 비용_농침:,} 만원")
print(f"헛걸음 {fp}건 = {fp * 비용_헛걸음:,} 만원")
print(f"합계 = {손실:,} 만원")
```

- ▶ 농침 18 건 = 36,000 만원
- ▶ 헛걸음 9 건 = 270 만원
- ▶ 합계 = 36,270 만원

손실의 99.3%가 농침 한 쪽에서 나옵니다. 헛걸음을 9 건에서 0 건으로 줄여 봐야 270 만 원이지만, 농침을 18 건에서 12 건으로만 줄여도 1 억 2 천만 원입니다. 이 비율을 한 번 계산해 보면 왜 재현율부터 봐야 하는지가 설득이 아니라 산수가 됩니다.

설비 적용

실제 프로젝트에서는 이 두 단가를 현장에서 받아 와야 합니다. 정비 이력에서 계획 외 정지 1 회의 평균 손실과 점검 1 회의 표준 공수를 뽑으면 됩니다. 단가가 손에 있으면 모델 개선의 목표가 "정확도 1%p 올리기"가 아니라 "연간 손실 얼마 줄이기"가 되고, 그때부터 예산 이야기를 할 수 있게 됩니다.

주의

TN 이 압도적으로 많은 것 자체는 좋은 일이지만, 한 가지 부작용이 있습니다. **고장 데이터가 계속 적게 쌓입니다.** 다음에 모델을 다시 학습시킬 때 고장 사례가 여전히 부족하면 모델은 좀처럼 나아지지 않습니다. 그래서 신규 설비이거나 고장 이력이 없는 라인에서는 **고장 쪽 데이터를 의도적으로 확보하는 계획**이 따로 필요합니다.

2 장 한 장 정리

배운 것	한 줄 정의	기억할 숫자
FN 농침	고장인데 정상이라 함 — 경고 없는 정지	18 대 · 가장 먼저 줄일 것
FP 헛걸음	정상인데 고장이라 함 — 출동 비용과 신뢰 손실	9 대 · 0 이 목표가 아님
농침 18 건의 확률	0.06 ~ 0.49, 절반 이상이 경계 바로 아래	0.3 이상이 10 건
비용으로 환산	틀림의 종류마다 단가가 다르다	손실의 99.3%가 농침
현장 언어 번역	통계 용어를 대수·금액으로 옮긴다	읽는 사람에 따라 두 번 번역

한 장 정리

2 장의 결론은 **틀림에도 값이 매겨져 있다**는 것입니다. 27 건을 뭉뚱그려 보면 아무 판단도 안 나오지만, 18 건과 9 건으로 갈라 놓고 단가를 곱하면 무엇부터 고쳐야 할지가 한 줄로 나옵니다. 다음 장에서는 이 개수를 **비율**로 바꿉니다.

혼동행렬의 네 칸은 **절대 개수**입니다. 우리 공장은 200 대를 평가했고, 옆 공장은 8 대만 평가했다고 해 봅시다. 우리 농침이 18 건이고 옆 공장이 2 건이라면 어느 쪽이 더 나쁜 걸까요. 개수만으로는 답할 수 없습니다. 시험 문제 수가 다르면 맞힌 개수를 비교해도 소용없는 것과 같습니다.

그래서 **비율**이 필요합니다. 그런데 무엇을 분모로 둘 것인가에 따라 전혀 다른 두 개의 비율이 나옵니다. 그 둘이 오늘의 주인공인 정밀도와 재현율입니다.

3.1 분모를 무엇으로 두느냐

지표	공식	분모가 뜻하는 것	묻는 질문
정밀도	$TP / (TP + FP)$	모델이 고장이라고 답한 전부	내가 경고한 것 중 진짜는 얼마였나
재현율	$TP / (TP + FN)$	실제로 고장이었던 전부	실제 고장 중 내가 잡은 건 얼마였나

분자는 둘 다 TP로 같습니다. **다른 것은 분모뿐**입니다. 정밀도의 분모에는 FP가, 재현율의 분모에는 FN이 들어갑니다. 그리고 2장에서 본 대로 **FN이 비싼 쪽**이므로, 설비 예측에서 먼저 보는 것은 재현율입니다. 이 연결 고리 하나만 잡아 두면 나머지가 다 따라옵니다.

개념

분모에 무엇이 들어 있는지를 보십시오. 정밀도의 분모에는 FP(헛걸음)가 들어 있으니 헛걸음이 늘면 정밀도가 떨어집니다. 재현율의 분모에는 FN(농침)이 들어 있으니 농침이 늘면 재현율이 떨어집니다. 그래서 **FN → 재현율 → 설비 비용**이 하나의 사슬로 이어집니다.

3.2 손으로 계산해 보기

공식이 손에 익어야 함수 결과를 믿을 수 있습니다. 네 칸을 직접 대입해 봅시다.

```
tn, fp, fn, tp = cm.ravel()

exp_precision = tp / (tp + fp)      # 26 / (26 + 9)
exp_recall    = tp / (tp + fn)      # 26 / (26 + 18)

print(f"정밀도: {round(exp_precision, 3)}")
print(f"재현율: {round(exp_recall, 3)}")

▶ 정밀도: 0.743
▶ 재현율: 0.591
```

정밀도 0.743 — 고장이라고 경고한 35대 중 26대가 진짜였습니다. 열 번 경고하면 일곱 번은 맞습니다. **재현율 0.591** — 실제 고장 44대 중 26대만 잡았습니다. **열 대 중 네 대를 그냥 흘려보냈다는 뜻**입니다.

주의

두 값을 계산하기 전에 `cm.ravel()`을 **다시 한 번** 실행해 네 변수를 채웠습니다. 앞에서 이미 꺼냈더라도, 중간에 다른 셀에서 같은 이름을 덮어썼을 수 있기 때문입니다. 노트북은 셀을 위아래로 옮겨 다니며 실행하게 되므로 **지표를 계산하기 직전에 네 칸을 다시 꺼내는 습관**이 안전합니다.

3.3 함수로 구하기

함수 precision_score() 정밀도 — 경고의 신뢰도

정의 모델이 1 이라고 답한 것 중 실제로 1이었던 비율을 돌려줍니다. 공식을 몰라도 값이 나옵니다.

형태 precision_score(y_test, y_pred)

손으로 계산한 값이 맞는지 함수로 대조합니다.

```
from sklearn.metrics import precision_score, recall_score

skl_prec = precision_score(cy_test, cy_pred)
skl_recall = recall_score(cy_test, cy_pred)
```

```
print("정밀도:", skl_prec)
print("재현율:", skl_recall)
```

```
▶ 정밀도: 0.7428571428571429
▶ 재현율: 0.5909090909090909
```

손계산과 소수점 끝까지 같습니다. 0.743 과 0.591 은 반올림한 값이었을 뿐입니다. 공식을 제대로 이해했다는 확인이자, 함수를 믿어도 된다는 확인입니다.

왜 쓰나 매번 ravel 로 네 칸을 꺼내 나누는 것은 순서를 틀릴 여지가 있습니다. 함수는 정답과 예측만 넘기면 되므로 그 사고가 원천적으로 생기지 않습니다. 공식은 이해용, 함수는 실무용입니다.

응용 라벨이 0 과 1 이면 1 을 양성으로 잡고 자동 계산합니다. 라벨이 다른 값이거나 양성을 다른 쪽으로 두고 싶으면 pos_label 로 지정합니다.

```
# 라벨이 "정상"/"고장" 같은 문자열일 때는 어느 쪽이 양성인지 알려 줘야 합니다
precision_score(y_text_test, y_text_pred, pos_label="고장")
```

주의

인자 순서는 여기서도 (정답, 예측)입니다. accuracy_score 는 순서를 바꿔도 값이 같았지만 정밀도와 재현율은 바꾸면 서로의 값이 됩니다. 순서를 뒤집어 넣으면 정밀도 자리에 재현율 값이 조용히 나옵니다.

함수 recall_score() 재현율 — 놓치지 않는 능력

정의 실제로 1 이었던 것 중 모델이 1 이라고 잡아낸 비율을 돌려줍니다.

형태 recall_score(y_test, y_pred)

이름이 낯설지만 뜻은 단순합니다. 재현(再現)은 같은 것을 다시 나타낸다는 말입니다.

```
print("실제 고장 대수:", int(cy_test.sum()))
print("그중 잡아낸 대수:", int(((cy_pred == 1) & (cy_test.values == 1)).sum()))
print("재현율:", recall_score(cy_test, cy_pred))
```

```
▶ 실제 고장 대수: 44
▶ 그중 잡아낸 대수: 26
▶ 재현율: 0.5909090909090909
```

$26 \div 44 = 0.591$ 입니다. 재현율은 실제 고장 목록을 손에 들고 몇 개에 동그라미가 쳐졌는지 세는 것과 같습니다.

왜 쓰나 설비 예측에서 이 값이 1 순위인 이유는 단순합니다. 분모에 FN 이 들어 있기 때문입니다. 놓침이 늘면 재현율이 그만큼 떨어지므로, 재현율 하나만 추적해도 가장 비싼 실수를 감시하는 셈이 됩니다.

개념

AI 모델의 결과가 매번 똑같지 않은 것도 같은 뿌리의 이야기입니다. 같은 프롬프트를 넣어도 챗봇 답이 조금씩 달라지는 것처럼, 모델은 확률로 답을 냅니다. 얼마나 일관되게 같은 답을 재현하는가가 재현이라는 말에 담긴 감각입니다.

설비 적용

현장 보고에서는 "재현율 59.1%"라고 쓰지 말고 "실제로 고장이 임박했던 44 대 중 26 대에 미리 경고를 냈습니다"라고 쓰십시오. 같은 값인데 받아들이는 무게가 다릅니다.

3.4 한 쌍의 안경 — 한쪽만 보면 왜곡됩니다

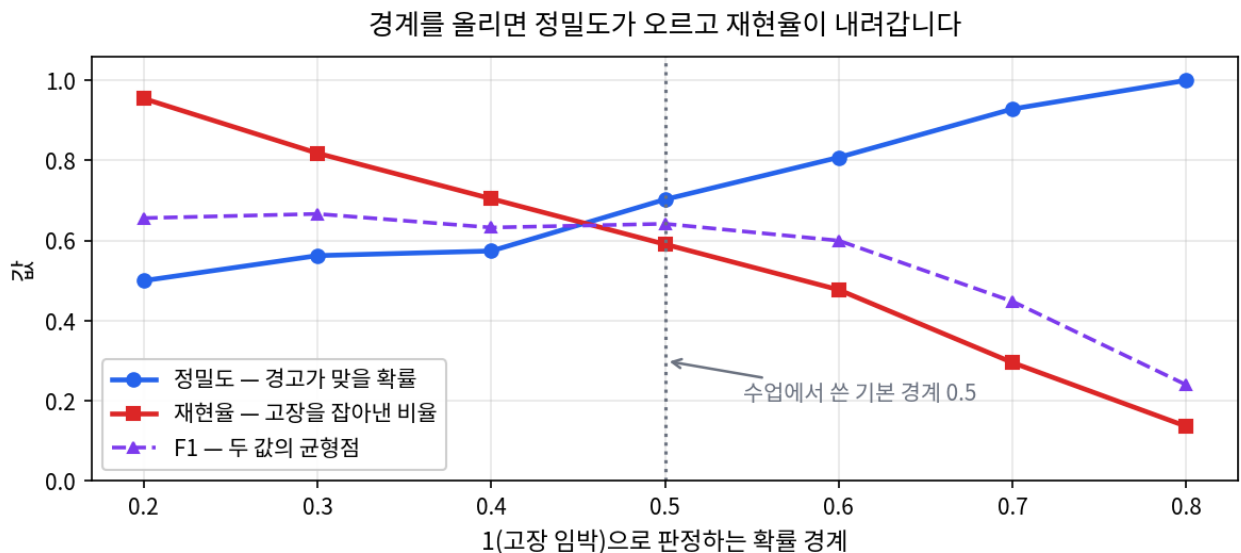
두 값은 **한쪽을 올리면 다른 쪽이 내려가는** 관계입니다. 억지로 만든 관계가 아니라 구조상 그렇습니다. 경고를 남발하면 실제 고장은 대부분 걸리지만(재현율 ↑) 헛걸음이 늘고(정밀도 ↓), 확실할 때만 경고하면 맞는 말만 하지만(정밀도 ↑) 애매한 고장을 흘려보냅니다(재현율 ↓).

말로만 하면 와닿지 않으니, 38 일차에 배운 `predict_proba`로 **판정 경계를 직접 옮겨 가며** 두 값을 재 봤습니다.

```
proba = c_model.predict_proba(cX_test)[: , 1]

for 경계 in [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    판정 = (proba >= 경계).astype(int)
    p = precision_score(cy_test, 판정, zero_division=0)
    r = recall_score(cy_test, 판정)
    print(f"경계 {경계}   정밀도 {p:.3f}   재현율 {r:.3f}")
```

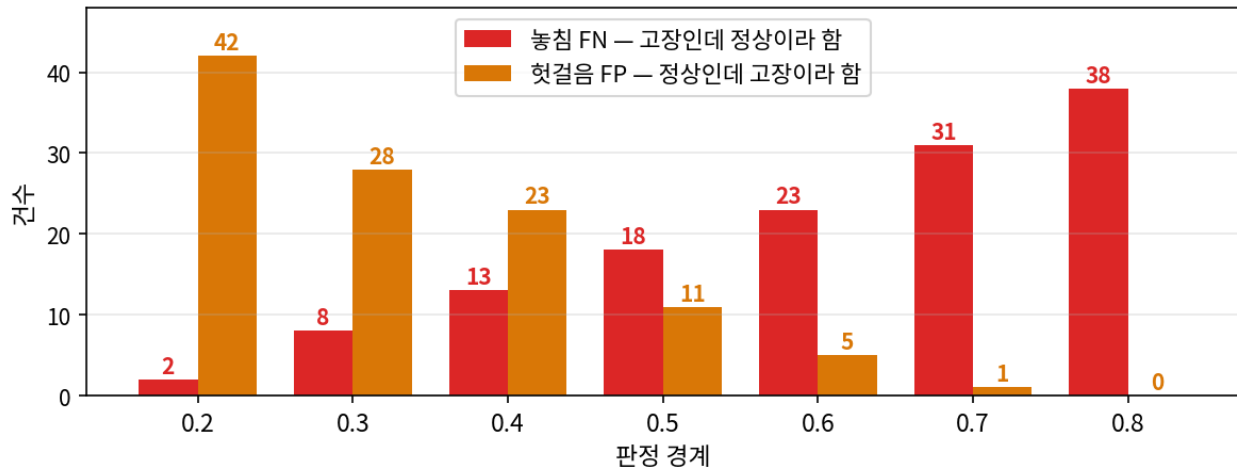
- ▶ 경계 0.2 정밀도 0.500 재현율 0.955
- ▶ 경계 0.3 정밀도 0.562 재현율 0.818
- ▶ 경계 0.4 정밀도 0.574 재현율 0.705
- ▶ 경계 0.5 정밀도 0.703 재현율 0.591
- ▶ 경계 0.6 정밀도 0.808 재현율 0.477
- ▶ 경계 0.7 정밀도 0.929 재현율 0.295
- ▶ 경계 0.8 정밀도 1.000 재현율 0.136



▲ 파란 선이 정밀도, 빨간 선이 재현율 — 두 선이 반대로 움직이고 0.45 부근에서 교차합니다

그림이 말을 대신합니다. **경계를 올리면 파란 선이 올라가고 빨간 선이 내려갑니다.** 경계 0.8에서는 정밀도가 1.000입니다 — 경고한 6대가 전부 진짜 고장이었습니다. 완벽해 보이지만 **44대 중 38대를 놓쳤습니다.** 반대로 경계 0.2에서는 재현율 0.955로 거의 다 잡았지만, 경고한 84대 중 절반이 헛걸음이었습니다.

경계를 낮추면 놓침이 줄고 헛걸음이 늘니다 — 공짜인 지점은 없습니다



▲ 같은 실험을 건수로 본 것 — 경계를 낮추면 빨강(놓침)이 줄고 주황(헛걸음)이 늘니다

건수로 보면 맞바꿈이 더 분명합니다. 경계 0.3에서 **놓침이 18건에서 8건으로 줄지만 헛걸음은 9건에서 28건으로 늘어납니다**. 2장의 단가를 그대로 곱해 보면 손실이 36,270 만 원에서 16,840 만 원으로 떨어집니다 — **모델을 다시 학습시키지 않고 숫자 하나만 바꿔서 얻는 결과입니다**.

설비 적용

그래서 실무의 질문은 "정밀도와 재현율 중 뭐가 더 중요한가"가 아니라 "**우리 현장에서 어느 실수가 더 비싼가**"입니다. 항공 보안 검색은 위험물을 놓치는 쪽이 압도적으로 비싸므로 기준을 깐깐하게 잡습니다 — 재현율을 올리는 대신 멀쩡한 승객이 붙들리는 정밀도 하락을 감수하는 것입니다. 설비 예측도 같은 자리에 있습니다.

주의

위 표의 경계 0.5 줄(정밀도 0.703)이 앞에서 구한 0.743 과 다릅니다. **확률이 정확히 0.50 인 행이 2 건** 있는데, `proba >= 0.5` 는 이 둘을 1로 넣고 `predict` 는 0으로 처리하기 때문입니다. 둘 다 틀린 것이 아니라 **동점을 처리하는 규칙이 다를 뿐입니다**.

3 장 한 장 정리

배운 것	한 줄 정의	기억할 숫자
<code>precision_score</code>	고장이라 경고한 것 중 진짜였던 비율	0.743 · 35 대 중 26 대
<code>recall_score</code>	실제 고장 중 잡아낸 비율	0.591 · 44 대 중 26 대
분모의 차이	정밀도는 FP, 재현율은 FN 이 분모에 들어간다	FN → 재현율 → 비용
트레이드오프	한쪽을 올리면 다른 쪽이 내려간다	경계 0.8에서 정밀도 1.000, 재현율 0.136
경계 조정	학습을 다시 하지 않고 숫자 하나로 바꾼다	0.3 이면 놓침 18→8 건

한 장 정리

3 장의 결론은 **두 값은 한 쌍의 안경**이라는 것입니다. 한쪽 알만 끼면 세상이 기울어 보입니다. 정밀도만 보면 경고를 아끼는 모델이 좋아 보이고, 재현율만 보면 아무 데나 경고를 때리는 모델이 좋아 보입니다. 다음 장에서 정확도까지 셋을 나란히 놓습니다.

이제 정확도·정밀도·재현율 셋을 한 번에 찍어 봅니다. 수업의 마지막 코드였고, 실무에서 리포트의 첫 줄에 들어가는 형태이기도 합니다.

4.1 세 줄 출력

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

acc = accuracy_score(cy_test, cy_pred)
prec = precision_score(cy_test, cy_pred)
rec = recall_score(cy_test, cy_pred)

print(f"정확도: {acc:.3f}")
print(f"정밀도: {prec:.3f}")
print(f"재현율: {rec:.3f}")
```

- ▶ 정확도: 0.865
- ▶ 정밀도: 0.743
- ▶ 재현율: 0.591

같은 모델, 같은 200 대인데 숫자가 0.865 에서 0.591 까지 벌어집니다. 어느 것도 거짓이 아닙니다. 세 숫자는 서로 다른 질문에 답하고 있을 뿐입니다.

지표	값	이 숫자가 답하는 질문	분모
정확도	0.865	200 대 전체 중 몇 대를 맞혔나	전체 200 대
정밀도	0.743	고장이라 경고한 35 대 중 몇 대가 진짜였나	경고한 35 대
재현율	0.591	실제 고장 44 대 중 몇 대를 잡았나	실제 고장 44 대

정확도가 가장 높은 이유는 **분모가 가장 관대하기 때문**입니다. 200 대 중 156 대가 원래 정상이고, 정상은 맞히기 쉽습니다. 정확도는 그 쉬운 문제 156 개를 포함해서 채점한 점수입니다. 재현율은 **어려운 문제 44 개만 골라서** 채점한 점수입니다.

4.2 더미와 다시 붙여 보기

38 일차에 무조건 정상이라고만 찍는 더미 모델을 만들었고, 정확도 0.78 이 나와 우리 모델의 0.865 와 큰 차이가 없어 보였습니다. 이제 세 지표로 다시 재 봅니다.

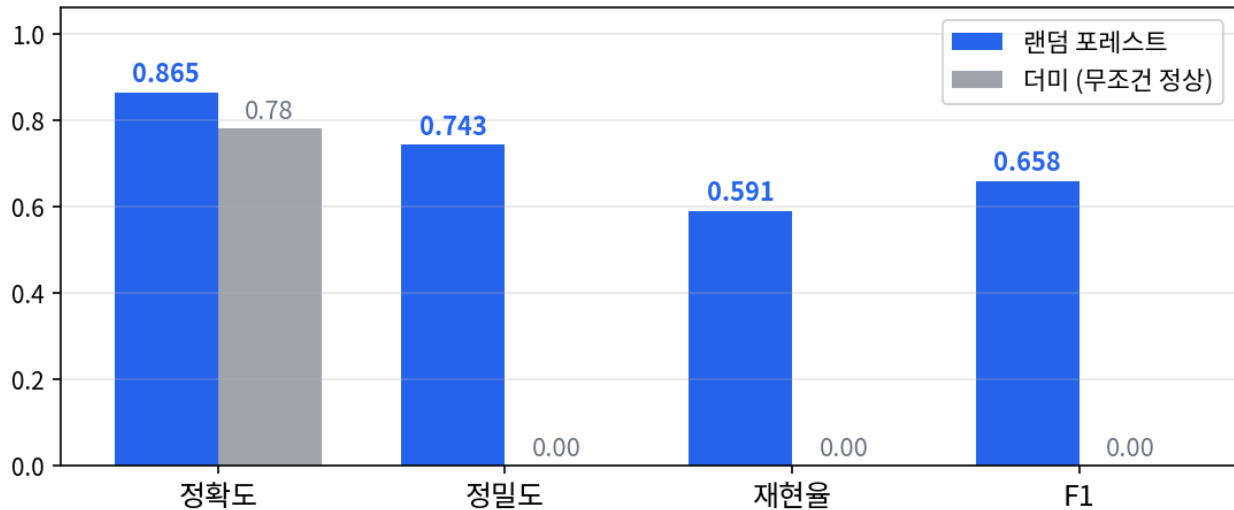
```
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(strategy="most_frequent").fit(cX_train, cy_train)
dm_pred = dummy.predict(cX_test)

print("더미 정확도:", accuracy_score(cy_test, dm_pred))
print("더미 재현율:", recall_score(cy_test, dm_pred))
print("더미 정밀도:", precision_score(cy_test, dm_pred, zero_division=0))
```

- ▶ 더미 정확도: 0.78
- ▶ 더미 재현율: 0.0
- ▶ 더미 정밀도: 0.0

정확도만 보면 0.865 대 0.78 — 재현율로 보면 0.591 대 0



▲ 정확도로 보면 0.865 대 0.78 로 8.5%p 차이지만, 재현율로 보면 0.591 대 0 — 같은 두 모델입니다

더미의 재현율은 0 입니다. 당연합니다. 1 이라고 답한 적이 한 번도 없으니 잡아낸 고장도 0 건입니다. 정확도로 보면 우리 모델이 겨우 8.5%p 앞서는 것 같았지만, **고장을 잡는 능력으로 보면 0 건 대 26 건**입니다. 비교 자체가 성립하지 않는 차이였습니다.

개념

38 일차에 남았던 질문이 여기서 풀립니다. 그때는 "8.5%p 차이가 의미 있는 수준인가"에서 판단이 애매했습니다. **애매했던 이유는 지표를 잘못 골랐기 때문**이었습니다. 불균형 데이터에서 정확도는 다수 클래스의 점수를 그대로 베풀어 줍니다. 지표를 바꾸자 같은 결과가 전혀 다르게 보입니다.

주의

더미의 정밀도에 `zero_division=0`을 넣었습니다. 정밀도의 분모는 TP+FP 인데 더미는 **1 이라고 답한 적이 없어 분모가 0**이 됩니다. 이 옵션이 없으면 경고가 뜨면서 값이 0 으로 채워집니다. 교안에는 없는 옵션이지만, 더미처럼 한쪽만 찍는 모델을 평가할 때는 넣어 주는 편이 깔끔합니다.

4.3 치우침을 한 문장으로

세 숫자를 나열하고 끝내면 읽는 사람이 판단을 못 합니다. **어느 쪽으로 치우쳤는지 한 문장**을 붙여 주는 것까지가 마무리입니다. 판단 규칙은 단순합니다.

패턴	뜻하는 것	해야 할 일
정확도 높음 · 재현율 낮음	다수 클래스(정상)만 잘 맞히는 중입니다. 불균형에 속고 있습니다.	고장 쪽 학습 데이터를 늘리거나 판정 경계를 낮춥니다.
재현율 높음 · 정밀도 낮음	웬만하면 다 경고합니다. 적극적인 모델입니다.	헛걸음 비용이 감당되는지 확인하고, 안 되면 경계를 올립니다.
둘 다 높음	균형이 잡혔습니다. 드뭅니다.	현 상태를 기준으로 기록해 둡니다.
둘 다 낮음	입력이나 라벨에 문제가 있을 가능성이 큼니다.	모델보다 데이터를 먼저 의심합니다.

우리 모델은 첫 번째 줄입니다. 정확도 0.865 는 높는데 재현율 0.591 이 낮습니다. 그래서 한 문장으로 옮기면 이렇게 됩니다.

설비 적용

이 모델은 실제 고장 44 대 중 26 대(59.1%)에 미리 경고를 냈고, 경고 35 건 중 26 건(74.3%)이 실제 고장이었습니다. 전체 정확도는 86.5%지만 놓친 18 대가 남아 있어 현장 투입 전 재현율 개선이 필요합니다. 보고서 첫 줄에 이 한 문장을 넣고, 그 아래에 혼동행렬과 세 숫자를 붙이면 됩니다. 숫자만 보낸 표와 읽히는 정도가 다릅니다.

주의

재현율을 먼저 본다는 말이 **정밀도는 대충 봐도 된다는** 뜻은 아닙니다. 재현율만 쫓다 보면 웬만한 건 다 경고하는 모델이 되고, 그러면 현상이 경고를 믿지 않게 됩니다. **우선순위를 둔다는 것이지 하나를 버린다는 것이 아닙니다.** 균형이 무너지면 결국 둘 다 잃습니다.

4.4 네 모델을 한 표로 세워 보기

지표 셋을 손에 넣었으니 이제 **모델끼리 줄을 세울 수 있습니다.** 같은 학습·평가 데이터로 성격이 다른 네 모델을 돌려 봤습니다.

```
from sklearn.tree import DecisionTreeClassifier

models = {
    "랜덤 포레스트": RandomForestClassifier(n_estimators=100, random_state=42),
    "결정 트리 한 그루": DecisionTreeClassifier(random_state=42),
    "얇은 트리(깊이 3)": DecisionTreeClassifier(max_depth=3, random_state=42),
    "더미(무조건 정상)": DummyClassifier(strategy="most_frequent"),
}

for 이름, m in models.items():
    m.fit(cX_train, cy_train)
    p = m.predict(cX_test)
    tn, fp, fn, tp = confusion_matrix(cy_test, p).ravel()
    print(f"{이름:16s} 정확도 {accuracy_score(cy_test, p):.3f} "
          f"정밀도 {precision_score(cy_test, p, zero_division=0):.3f} "
          f"재현율 {recall_score(cy_test, p):.3f} 놓침 {fn:2d} 헛걸음 {fp:2d}")
```

- ▶ 랜덤 포레스트 정확도 0.865 정밀도 0.743 재현율 0.591 놓침 18 헛걸음 9
- ▶ 결정 트리 한 그루 정확도 0.785 정밀도 0.512 재현율 0.477 놓침 23 헛걸음 20
- ▶ 얇은 트리(깊이 3) 정확도 0.825 정밀도 0.605 재현율 0.591 놓침 18 헛걸음 17
- ▶ 더미(무조건 정상) 정확도 0.780 정밀도 0.000 재현율 0.000 놓침 44 헛걸음 0

표로 정리하면 각 모델의 성격이 드러납니다.

모델	정확도	정밀도	재현율	놓침	헛걸음	한 줄 평
랜덤 포레스트	0.865	0.743	0.591	18	9	네 지표 모두 1 위입니다.
결정 트리 한 그루	0.785	0.512	0.477	23	20	더미보다 정확도가 겨우 0.5%p 높습니다.
얇은 트리(깊이 3)	0.825	0.605	0.591	18	17	재현율이 숲과 같습니다.
더미(무조건 정상)	0.780	0.000	0.000	44	0	44 대를 전부 놓쳤습니다.

두 줄이 눈에 걸립니다. 첫째, **결정 트리 한 그루는 정확도 0.785로 더미(0.780)와 거의 같습니다.** 정확도만 봤다면 학습이 안 됐다고 판단했겠지만, 재현율 0.477을 보면 **고장의 절반 가까이를 실제로 잡고 있습니다.** 정확도가 못 보여 주는 것을 재현율이 보여 준 경우입니다.

둘째, **얇은 트리는 랜덤 포레스트와 재현율이 0.591로 똑같습니다.** 놓침 18대도 같습니다. 그런데 헛걸음이 9대와 17대로 갈립니다. **두 모델은 같은 고장을 잡고 다른 헛걸음을 냅니다.** 재현율 하나만 보고 골랐다면 둘을 구분하지 못했을 것입니다.

설비 적용

이 표가 실무에서 모델을 고르는 자리입니다. **놓침이 같다면 헛걸음이 적은 쪽을 고르고,** 놓침이 다르면 2장의 단가를 곱해 손실이 작은 쪽을 고릅니다. 얇은 트리의 손해는 헛걸음 8대분 = 240만 원입니다 — 대신 얇은 트리는 export_text로 판단 근거를 글자로 꺼내 볼 수 있습니다. **설명 가능성이 필요한 자리라면 240만 원이 그 대가인 셈입니다.**

주의

네 모델을 같은 `random_state`, 같은 학습·평가 분할로 돌렸기 때문에 비교가 성립합니다. 분할이 다르면 숫자 차이가 모델 차이인지 데이터 차이인지 알 수 없습니다. 여러 모델을 비교할 때는 `train_test_split`을 한 번만 실행하고 그 결과를 돌려쓰는 것이 원칙입니다.

4 장 한 장 정리

배운 것	한 줄 정의	기억할 숫자
세 지표 한 화면	같은 모델을 서로 다른 분모로 채점한 결과	0.865 · 0.743 · 0.591
정확도가 가장 높은 이유	분모에 쉬운 문제(정상 156 대)가 다 들어 있다	전체 200 대 실제고장 44
더미와의 재대결	정확도 8.5%p 차이가 재현율로는 0 대 26 건	더미 재현율 0.000
<code>zero_division=0</code>	분모가 0 일 때 경고 대신 0 을 돌려준다	더미의 정밀도 계산에 필요
한 문장 진단	정확도 ↑ 재현율 ↓ = 불균형에 속는 중	놓친 18 대가 과제

한 장 정리

4 장의 결론은 **지표를 바꾸면 같은 결과가 전혀 다르게 보인다**는 것입니다. 38 일차의 8.5%p 는 사실 0 건 대 26 건이었습니다. 그러니 리포트에 숫자 하나만 심지어 말고, **셋을 나란히 놓고 치우침을 한 문장으로 적으십시오.**

수업 마지막에 두 가지가 이름만 지나갔습니다. 정밀도와 재현율을 한 숫자로 묶는 **F1**, 그리고 세 지표를 클래스별로 한 표에 찍어 주는 **classification_report** 입니다. 코드로는 돌리지 않았지만 곧 쓰게 되므로 여기서 정리해 둡니다.

5.1 F1 — 두 값을 한 숫자로

함수 `f1_score()` 정밀도와 재현율의 조화평균

정의 두 값을 한쪽이 낮으면 전체가 확 떨어지는 방식으로 묶은 하나의 점수입니다.

형태 `f1 = 2 * (정밀도 * 재현율) / (정밀도 + 재현율)`

모델을 여러 개 비교할 때 숫자 두 개씩 들고 다니면 순위를 매기기 어렵습니다. 하나로 줄이는 방법이 필요합니다.

```
from sklearn.metrics import f1_score

손계산 = 2 * (skl_prec * skl_recall) / (skl_prec + skl_recall)

print("손계산 F1:", round(손계산, 4))
print("함수 F1:", round(f1_score(cy_test, cy_pred), 4))
```

▶ 손계산 F1: 0.6582
▶ 함수 F1: 0.6582

0.743 과 0.591 의 F1 이 **0.658** 입니다. 두 값의 산술평균(0.667)보다 조금 낮습니다. 조화평균은 낮은 쪽에 더 끌려가기 때문입니다.

왜 쓰나 그냥 평균을 쓰면 정밀도 1.0 에 재현율 0.0 인 극단적인 모델도 0.5 가 나와 그럴듯해 보입니다. **조화평균은 그런 모델에 0 을 줍니다.** 한쪽이 0 이면 분자가 0 이 되기 때문입니다. 균형을 강제하는 장치인 셈입니다.

응용 3 장에서 경계를 옮겨 가며 전 표에 F1 을 같이 놓으면, **두 값이 가장 잘 균형 잡히는 경계**가 어디인지 한눈에 보입니다.

```
for 경계 in [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    판정 = (proba >= 경계).astype(int)
    print(f"경계 {경계}   F1 {f1_score(cy_test, 판정):.3f}")
```

▶ 경계 0.2 F1 0.656
▶ 경계 0.3 F1 0.667
▶ 경계 0.4 F1 0.633
▶ 경계 0.5 F1 0.642
▶ 경계 0.6 F1 0.600
▶ 경계 0.7 F1 0.448
▶ 경계 0.8 F1 0.240

주의

F1 이 가장 높은 경계가 곧 현장에 맞는 경계는 아닙니다. 위 표에서는 0.3 이 F1 최고점이지만, F1 은 농침과 헛걸음을 같은 무게로 취급합니다. 2 장에서 본 것처럼 실제 단가는 예수 배 넘게 차이 날 수 있으므로, **F1 은 모델끼리 비교할 때 쓰고 운영 경계는 비용으로 정하십시오.** 위 표의 0.5 줄이 본문의 0.658 과 다른 것은 3 장 끝에서 짚은 동점 2 건 때문입니다.

5.2 classification_report — 한 표로

함수 `classification_report()` 세 지표를 클래스별로 한 표에

정의 정밀도·재현율·F1 을 클래스마다 각각 계산해 표 형태 문자열로 돌려줍니다.

형태 `print(classification_report(y_test, y_pred))`

지금까지는 1(고장)에 대한 값만 봤습니다. 0(정상) 쪽 성적도 같이 보면 치우침이 더 분명해집니다.

```
from sklearn.metrics import classification_report

print(classification_report(cy_test, cy_pred,
                           target_names=["정상(0)", "고장임박(1)"], digits=3))
```

	precision	recall	f1-score	support
정상(0)	0.891	0.942	0.916	156
고장임박(1)	0.743	0.591	0.658	44
accuracy			0.865	200
macro avg	0.817	0.767	0.787	200
weighted avg	0.858	0.865	0.859	200

support 가 그 클래스의 실제 개수입니다. 156 과 44, 즉 우리가 계속 봐 온 그 숫자입니다. **정상 줄과 고장 줄을 위아래로 비교하면 치우침이 그대로 드러납니다** — 정상은 재현율 0.942 인데 고장은 0.591 입니다.

왜 쓰나 한 줄짜리 함수로 **지금까지 배운 모든 것이 한 표에** 나옵니다. 실무에서는 이걸 먼저 찍어 보고, 이상한 줄이 보이면 그때 혼동행렬을 열어 봅니다.

설비 적용

macro avg 는 두 클래스를 **같은 무게로** 평균 낸 값이고, weighted avg 는 **개수 비율로** 평균 낸 값입니다. 불균형 데이터에서는 weighted avg 가 다수 클래스를 따라가 좋아 보이므로, **소수 클래스를 봐야 하는 우리 상황에서는 macro avg 가 더 정직한 요약입니다. 0.865(weighted)와 0.787(macro)의 차이가 그 편향의 크기입니다.**

주의

target_names 는 한글 이름을 붙이는 옵션이고 digits 는 소수점 자리수입니다(기본 2 자리). 둘 다 교안에는 없지만 **보기 좋게 만들 뿐 값은 바꾸지 않습니다**. 다만 target_names 의 순서는 라벨 오름차순(0, 1)과 맞춰야 하고, 뒤집어 넣으면 이름만 바뀌어 표가 거짓말을 하게 됩니다.

5 장 한 장 정리

배운 것	한 줄 정의	기억할 숫자
f1_score	정밀도와 재현율의 조화평균 — 낮은 쪽에 끌려간다	0.658 (산술평균 0.667)
F1 을 쓰는 자리	모델끼리 순위를 매길 때	운영 경계는 비용으로 정한다
classification_report	세 지표를 클래스별로 한 표에 찍는다	정상 0.942 대 고장 0.591
support	그 클래스의 실제 개수	156 · 44
macro vs weighted	같은 무게 평균 대 개수 비율 평균	0.787 대 0.865

한 장 정리

5 장은 오늘 코드로 돌리지 않은 내용이라 가볍게 읽어도 됩니다. 다만 classification_report 한 줄이면 **오늘 다섯 장에서 배운 값이 전부 한 표에 나온다는** 사실은 기억해 두십시오. 손으로 계산해 본 경험이 있어야 그 표를 읽을 수 있습니다.

A.1 네 칸과 지표

이름	공식 / 코드	우리 값	한 줄 뜻
TN	<code>cm[0][0]</code>	147	정상을 정상이라 했습니다.
FP	<code>cm[0][1]</code>	9	정상인데 고장이라 했습니다 (헛걸음).
FN	<code>cm[1][0]</code>	18	고장인데 정상이라 했습니다 (놓침).
TP	<code>cm[1][1]</code>	26	고장을 고장이라 했습니다.
정확도	$(TP+TN)/\text{전체}$	0.865	전체 중 맞힌 비율.
정밀도	$TP/(TP+FP)$	0.743	경고한 것 중 진짜였던 비율.
재현율	$TP/(TP+FN)$	0.591	실제 고장 중 잡아낸 비율.
F1	$2PR/(P+R)$	0.658	두 값의 조화평균.

A.2 함수 한 줄 요약

함수	인자	돌려주는 것
<code>confusion_matrix(y_test, y_pred)</code>	(정답, 예측)	2×2 넘파이 배열
<code>cm.ravel()</code>	—	tn, fp, fn, tp 네 값
<code>ConfusionMatrixDisplay(cm).plot()</code>	<code>cmap · values_format</code>	색칠된 표 그림
<code>accuracy_score(y_test, y_pred)</code>	(정답, 예측)	전체 정확도 하나
<code>precision_score(y_test, y_pred)</code>	<code>pos_label · zero_division</code>	정밀도 하나
<code>recall_score(y_test, y_pred)</code>	(정답, 예측)	재현율 하나
<code>f1_score(y_test, y_pred)</code>	(정답, 예측)	F1 하나
<code>classification_report(y_test, y_pred)</code>	<code>target_names · digits</code>	클래스별 표 문자열

A.3 오늘 코드 전체 흐름

```
import pandas as pd, numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import (confusion_matrix, ConfusionMatrixDisplay,
                             accuracy_score, precision_score, recall_score)

# 1. 38 일차에서 만든 모델을 그대로 씁니다
cdf = pd.read_csv("../Data/28_cmapss_fd001_sample.csv", encoding="utf-8")
feats = ["sensor_2", "sensor_3", "sensor_4", "sensor_7", "sensor_11", "sensor_15"]
cX, cy = cdf[feats], cdf["failure_soon"]
cX_train, cX_test, cy_train, cy_test = train_test_split(
    cX, cy, test_size=0.2, random_state=42, stratify=cy)
c_model = RandomForestClassifier(n_estimators=100, random_state=42).fit(cX_train, cy_train)
cy_pred = c_model.predict(cX_test)

# 2. 네 칸 만들기
```

```

cm = confusion_matrix(cy_test, cy_pred)
tn, fp, fn, tp = cm.ravel()          # 순서 엄수
print(cm, "   합:", tn + fp + fn + tp)

# 3. 현장 언어로
print(f"제대로 잡은 고장 {tp}대 · 놓친 고장 {fn}대 · 헛걸음 {fp}대")

# 4. 세 지표
print(f"정확도 {accuracy_score(cy_test, cy_pred):.3f}")
print(f"정밀도 {precision_score(cy_test, cy_pred):.3f}")
print(f"재현율 {recall_score(cy_test, cy_pred):.3f}")

```

```

▶ [[147   9]
   [ 18  26]]   합: 200
▶ 제대로 잡은 고장 26 대 · 놓친 고장 18 대 · 헛걸음 9 대
▶ 정확도 0.865
▶ 정밀도 0.743
▶ 재현율 0.591

```

B.1 ravel 연팩 순서 — 오류 없이 값만 뒤바뀝니다

실습 노트북에서 잡힌 오류입니다. **파이썬은 아무 불평도 하지 않고, 숫자만 조용히 틀립니다.** 이 단원에서 가장 위험한 종류의 버그라 따로 떼어 정리합니다.

```
# x 노트북에 적힌 코드
tn, tp, fn, fp = cm.ravel()
print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")

# ✓ 올바른 순서
tn, fp, fn, tp = cm.ravel()
print(f"TN={tn}, FP={fp}, FN={fn}, TP={tp}")
```

▶ TN=147, FP=26, FN=18, TP=9 ← x FP와 TP가 맞바뀐 값
▶ TN=147, FP=9, FN=18, TP=26 ← ✓

ravel()은 표를 **왼쪽 위 → 오른쪽 위 → 왼쪽 아래 → 오른쪽 아래** 순서로 펴서 147, 9, 18, 26을 내놓습니다. 왼쪽의 변수 이름은 그 순서대로 붙을 뿐이므로, 두 번째 자리에 tp를 적으면 **9가 TP가 되고 26이 FP가 됩니다.** 배열은 이름을 모릅니다.

이 한 줄이 아래로 어디까지 번지는지 보면 심각성이 분명해집니다.

이 변수로 계산하면	x 틀린 값	✓ 맞는 값	읽는 사람이 받는 인상
tp/(tp+fp) 정밀도	0.257	0.743	경고 열 번에 두세 번만 맞는 모델로 보입니다.
tp/(tp+fn) 재현율	0.333	0.591	고장의 3분의 1만 잡는 모델로 보입니다.
(tp+tn)/200 정확도	0.780	0.865	하필 더미 모델의 점수와 같아집니다.

자주 하는 실수

세 번째 줄이 특히 고약합니다. 틀린 정확도가 **0.780**인데, 이걸 38 일차에 구한 더미 베이스라인과 정확히 같은 값입니다. 그대로 보고했다면 "모델이 더미와 똑같다, 학습이 전혀 안 됐다"는 결론이 나옵니다. **멸절한 모델을 버리게 되는 사고입니다.**

주의

막는 방법은 두 가지입니다. 첫째, ravel 직후에 **네 칸의 합이 평가 대수와 같은지** 짚어 봅니다(합은 순서가 틀려도 같으니 이걸 못 잡습니다 — 그래서 둘째가 필요합니다). 둘째, **손계산 값을 precision_score·recall_score와 대조합니다.** 순서가 틀리면 두 값이 어긋나므로 바로 잡습니다. 3장에서 손계산과 함수를 굳이 나란히 짚어 본 이유가 이것입니다.

B.2 나머지 함정들

x 이렇게 하면	무슨 일이 일어나	✓ 바로잡기
confusion_matrix(y_pred, y_test)	표가 대각선 기준으로 뒤집혀 FN과 FP가 통째로 맞바뀝니다. 오류는 안 납니다.	항상 (정답, 예측) 순서로 넣습니다.
precision_score(y_pred, y_test)	정밀도 자리에 재현율 값이 나옵니다. 두 값이 통째로 바뀝니다.	평가 지표 함수는 모두 (정답, 예측)입니다.
정상을 양성으로 두기	정밀도·재현율이 전부 정상을 얼마나 잘 맞혔는지를 재계 됩니다. 값이 0.9 넘게 나와 좋아 보입니다.	찾아내려는 쪽이 양성입니다. 고장 임박이 1입니다.
disp.plot()만 하고 값 확인 안 함	그림은 나오지만 어느 칸이 FN인지 모른 채 색만 보게 됩니다.	그림과 함께 네 칸 숫자를 반드시 같이 짚습니다.

x 이렇게 하면	무슨 일이 일어나나	✓ 바로잡기
proba >= 0.5 와 predict 혼용	확률이 정확히 0.50 인 행의 처리가 달라 결과가 미세하게 어긋납니다.	한 문서 안에서는 한쪽 기준으로 통일합니다.
precision_score 를 한쪽만 찍는 모델에	분모가 0 이 되어 경고가 뜹니다.	zero_division=0 을 넣습니다.

C.1 제출 전 점검표

#	확인할 것	기준
1	네 칸의 합이 평가 대수와 같은가	$147+9+18+26 = 200$
2	ravel 인덱스 순서가 tn, fp, fn, tp 인가	부록 B.1
3	손계산 정밀도·재현율이 함수 결과와 일치하는가	$0.743 \cdot 0.591$
4	모든 지표 함수의 인자가 (정답, 예측) 순서인가	cy_test 가 먼저
5	여섯 대 예시의 네 칸이 2·1·1·2 로 나오는가	손으로 계산
6	세 지표를 한 화면에 출력했는가	정확도 · 정밀도 · 재현율
7	치우침을 한 문장으로 적었는가	4 장 3 절

C.2 스스로 답해 보기

질문	힌트
FN 과 FP 중 어느 쪽이 더 비싼지, 그 이유를 두 문장으로 말할 수 있는가	2 장 1·2 절
정확도 0.865 와 재현율 0.591 이 왜 이렇게 벌어지는지 분모로 설명할 수 있는가	4 장 1 절
더미 모델의 재현율이 0 인 이유를 한 줄로 말할 수 있는가	1 이라 답한 적이 없습니다
판정 경계를 0.3 으로 내리면 무엇이 줄고 무엇이 느는가	3 장 4 절 · 농침 18→8, 헛걸음 9→28
F1 이 산술평균보다 낮게 나오는 이유를 설명할 수 있는가	조화평균은 낮은 쪽에 끌려갑니다

C.3 다음 시간 예고 — 28_02 고장 예측 결과 해석

28 장의 마지막 챕터입니다. 오늘까지 만든 네 칸과 세 지표를 **현장에 낼 수 있는 형태의 결과물**로 정리하는 내용이 될 것으로 보입니다.

오늘 만든 것	다음 시간에 할 일로 이어지는 것
혼동행렬 네 칸	결과 해석 문단의 근거 표로 들어갑니다.
정밀도 · 재현율 · F1	모델의 성격을 한 문장으로 요약하는 재료가 됩니다.
classification_report	오늘 이름만 지나갔으니 실제로 돌려 보게 될 가능성이 큼니다.
판정 경계 조정	운영 기준선을 정하는 이야기로 확장됩니다.

연결

그리고 그 뒤에는 **SQL 과 실제 프로젝트 데이터**가 기다리고 있습니다. 오늘까지가 한 모델을 채점하는 법이었다면, 다음 단계는 훨씬 큰 원자료를 직접 열어 컬럼의 의미부터 파악하는 일입니다. 결측치와 이상치가 그대로 들어 있는 데이터라고 하니, **22~24 일차에 배운 전처리**가 다시 필요해집니다.

오늘 배운 것을 한 줄로 줄이면 이렇습니다. **하나였던 점수를 네 칸으로 쪼개고, 그 네 칸을 두 개의 비율로 다시 묶었습니다.** 그 과정에서 38 일차에 애매하게 남아 있던 질문 하나가 풀렸습니다.

오늘 할 수 있게 된 것

#	할 수 있게 된 것	근거가 되는 절
1	양성과 음성을 현장 기준으로 정할 수 있습니다.	0 장
2	혼동행렬을 만들고 네 칸을 정확한 순서로 꺼낼 수 있습니다.	1 장 1~3 절
3	네 칸을 정비팀이 읽는 문장으로 옮길 수 있습니다.	2 장 3 절
4	틀림의 종류마다 비용을 매겨 우선순위를 정할 수 있습니다.	2 장 4 절
5	정밀도와 재현율을 손으로도 함수로도 구할 수 있습니다.	3 장 2~3 절
6	판정 경계를 옮겨 두 값의 맞바꿈을 직접 만들 수 있습니다.	3 장 4 절
7	세 지표를 놓고 모델의 치우침을 한 문장으로 진단할 수 있습니다.	4 장 3 절

복습 루틴

언제	무엇을	얼마나
오늘 저녁	여섯 대 예시로 네 칸을 손으로 채워 보기	5 분
내일 수업 전	ravel 순서와 두 공식을 백지에 적어 보기	3 분
주말	판정 경계를 0.2~0.8로 바꿔 가며 세 지표를 다시 뽑아 보기	20 분
다음 주	자기 프로젝트 데이터에 혼동행렬을 그려 보기	30 분

한 장 정리

마지막으로 한 가지만 기억하십시오. **FN입니다.** 농침이 가장 비싸고, 농침은 재현율의 분모에 들어 있고, 그래서 설비 예측에서는 재현율부터 봅니다. 이 세 마디가 하나의 고리로 이어지면 오늘 배운 것은 다시 꺼내 쓸 수 있는 지식이 됩니다.